

CO3_LAB

#

```
=====
# NAIVE BAYES CLASSIFIER
# Confusion Matrix and Accuracy
# Google Colab
#
```

```
=====
# Step 1: Import Libraries
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score,
confusion_matrix, classification_report
```

#

```
=====
# Step 2: Create Dataset
#
```

```
=====
data = {
    "Age": [22, 25, 28, 35, 40, 45, 50, 55, 60, 65,
            23, 27, 32, 38, 43, 48, 53, 58, 63, 68],
    "Salary": [25000, 28000, 30000, 35000, 40000,
               45000, 50000, 55000, 60000, 65000,
```

```
26000, 29000, 33000, 38000, 43000,  
48000, 53000, 58000, 63000, 70000],
```

```
"Experience": [1, 2, 3, 5, 7, 9, 12, 15, 18, 20,  
1, 3, 4, 6, 8, 10, 13, 16, 19, 22],
```

```
"Purchased": [  
"No", "No", "No", "No", "Yes",  
"Yes", "Yes", "Yes", "Yes", "Yes",  
"No", "No", "No", "Yes", "Yes",  
"Yes", "Yes", "Yes", "Yes", "Yes"  
]  
}
```

```
df = pd.DataFrame(data)
```

```
print("DATASET")  
print("=" * 60)  
print(df)
```

```
#
```

```
=====
```

```
# Step 3: Separate Features and Target
```

```
#
```

```
=====
```

```
X = df[  
["Age", "Salary", "Experience"]  
]
```

```
y = df["Purchased"]
```

#

Step 4: Split Dataset into Training and Testing

#

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.30,  
    random_state=42,  
    stratify=y  
)
```

#

Step 5: Create Naïve Bayes Model

#

```
model = GaussianNB()
```

```
model.fit(  
    X_train,  
    y_train  
)
```

#

Step 6: Make Predictions

#

y_pred = model.predict(X_test)

#

Step 7: Display Predictions

#

print("\nPREDICTIONS")
print("=" * 60)

for actual, predicted in zip(y_test, y_pred):

print(
 f"Actual: {actual:5s} | "
 f"Predicted: {predicted}"
)

#

Step 8: Calculate Accuracy

#

```
=====
accuracy = accuracy_score(
    y_test,
    y_pred
)
```

```
print("\nACCURACY")
print("=" * 60)
```

```
print(
    f"Accuracy: {accuracy * 100:.2f}%"
)
```

#

```
=====
# Step 9: Confusion Matrix
```

#

```
=====
cm = confusion_matrix(
    y_test,
    y_pred,
    labels=["No", "Yes"]
)
```

```
print("\nCONFUSION MATRIX")
print("=" * 60)
```

```
print(cm)
```

#

```
=====
```

```
=====
```

```
# Step 10: Visualize Confusion Matrix
```

#

```
=====
```

```
plt.figure(figsize=(7, 5))
```

```
plt.imshow(cm)
```

```
plt.title("Naive Bayes Confusion Matrix")
```

```
plt.xlabel("Predicted Class")
```

```
plt.ylabel("Actual Class")
```

```
plt.xticks(  
    [0, 1],  
    ["No", "Yes"]  
)
```

```
plt.yticks(  
    [0, 1],  
    ["No", "Yes"]  
)
```

```
# Display values inside matrix
```

```
for i in range(2):
```

```
    for j in range(2):
```

```
        plt.text(  
            j,
```

```
        i,  
        cm[i, j],  
        ha="center",  
        va="center"  
    )
```

```
plt.colorbar()
```

```
plt.show()
```

```
#
```

```
=====
```

```
# Step 11: Classification Report
```

```
#
```

```
=====
```

```
print("\nCLASSIFICATION REPORT")  
print("=" * 60)
```

```
    print(  
classification_report(  
    y_test,  
    y_pred,  
    zero_division=0  
    )  
    )
```

```
#
```

```
=====
```

```
# Step 12: Test a New Sample
```

#

```
=====
print("\nNEW SAMPLE PREDICTION")
print("=" * 60)
```

```
new_person = [
    42,    # Age
    45000, # Salary
    8     # Experience
]
```

```
prediction = model.predict(
    new_person
)
```

```
probability = model.predict_proba(
    new_person
)
```

```
print(
    "Predicted Class:",
    prediction[0]
)
```

```
print(
    "Probability of No:",
    f"{probability[0][0] * 100:.2f}%"
)
```

```
print(
    "Probability of Yes:",
    f"{probability[0][1] * 100:.2f}%"
)
```

DATASET


```
=====
Age Salary Experience Purchased
0 22 25000 1 No
1 25 28000 2 No
2 28 30000 3 No
3 35 35000 5 No
4 40 40000 7 Yes
5 45 45000 9 Yes
6 50 50000 12 Yes
7 55 55000 15 Yes
8 60 60000 18 Yes
9 65 65000 20 Yes
10 23 26000 1 No
11 27 29000 3 No
12 32 33000 4 No
13 38 38000 6 Yes
14 43 43000 8 Yes
15 48 48000 10 Yes
16 53 53000 13 Yes
17 58 58000 16 Yes
18 63 63000 19 Yes
19 68 70000 22 Yes
```

PREDICTIONS

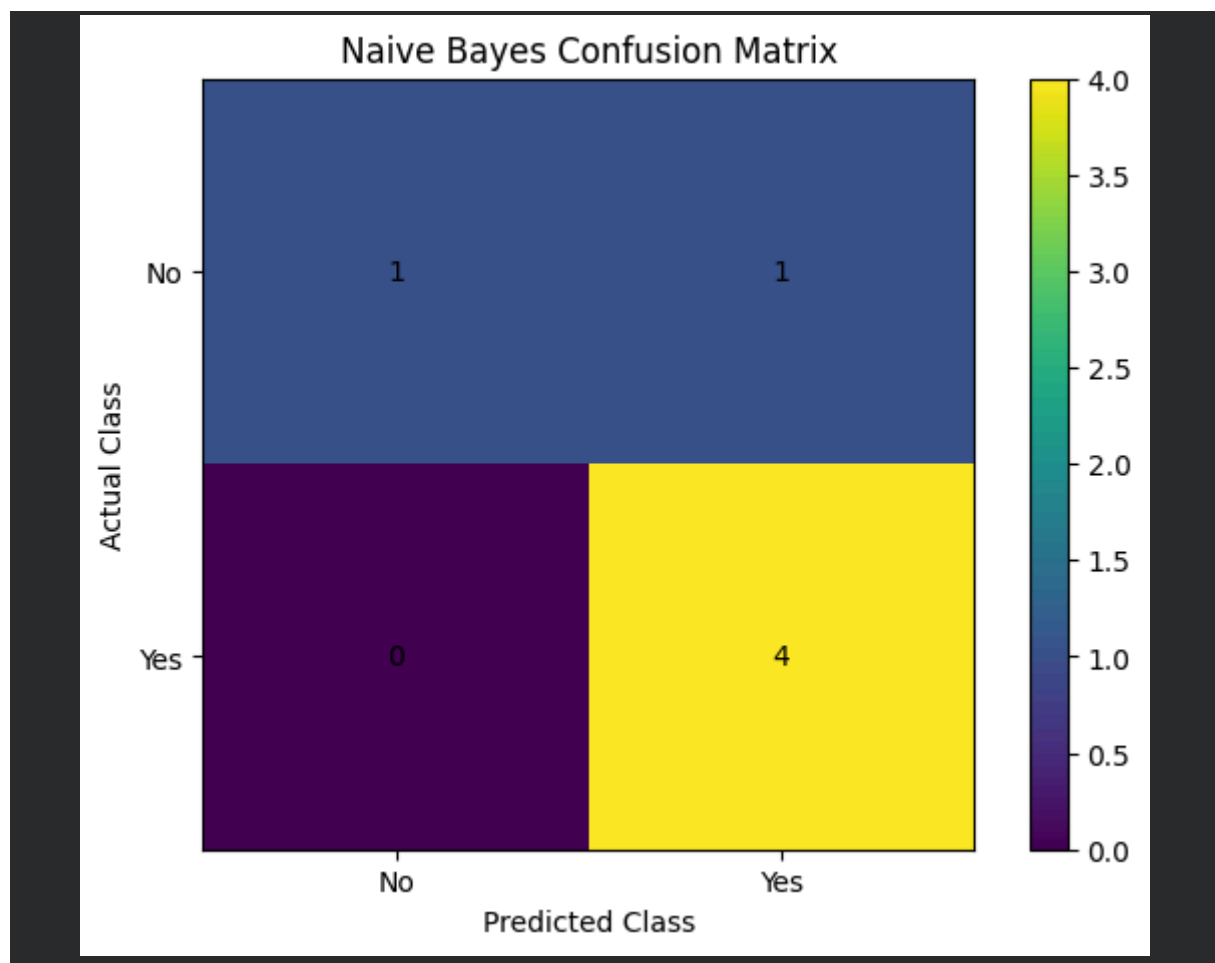
```
=====
Actual: No | Predicted: Yes
Actual: Yes | Predicted: Yes
Actual: No | Predicted: No
Actual: Yes | Predicted: Yes
Actual: Yes | Predicted: Yes
Actual: Yes | Predicted: Yes
```

ACCURACY

```
=====
Accuracy: 83.33%
```

CONFUSION MATRIX

```
=====
[[1 1]
 [0 4]]
```



CLASSIFICATION REPORT

=====

	precision	recall	f1-score	support
No	1.00	0.50	0.67	2
Yes	0.80	1.00	0.89	4
accuracy	0.83			6
macro avg	0.90	0.75	0.78	6
weighted avg	0.87	0.83	0.81	6